

LAPORAN TUGAS 2

SISTEM OPERASI



Dosen Pengampu Mata Kuliah:
Triawan Adi Cahyanto, M.Kom

Disusun Oleh:
Rafli Musthofa (2410651019)

PRODI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER
2025

DASAR TEORI

A. Konsep Proses

Proses adalah program yang sedang dieksekusi. Setiap proses memiliki *Process ID (PID)* unik yang digunakan oleh sistem operasi untuk mengatur, menjadwalkan, dan mengawasiproses.

Pada sistem operasi berbasis UNIX/Linux, proses baru dapat dibuat menggunakan system call `fork()`. Fungsi ini menduplikasi proses induk menjadi dua:

- Parent Process = proses yang memanggil `fork()`.
- Child Process = proses hasil duplikasi yang memiliki PID baru.

Hubungan antara parent dan child memungkinkan koordinasi pekerjaan, misalnya parent menunggu child selesai dengan `wait()`.

B. Sistem Manajemen Proses

Sistem operasi berperan sebagai pengelola proses (Process Manager). Ia mengatur pembuatan, eksekusi, dan terminasi proses.

Beberapa fungsi penting dalam manajemen proses:

- Process Creation: menggunakan `fork()` atau `execa()`.
- Synchronization: mengatur waktu dan urutan eksekusi antar proses.
- Interprocess Communication (IPC): memungkinkan proses saling bertukar data.

C. Interprocess Communication (IPC)

IPC merupakan mekanisme agar dua atau lebih proses dapat saling berkomunikasi. Dua metode populer:

1. Pipe/ FIFO : komunikasi satu arah atau dua arah antara proses. Data mengalir seperti pipa.
2. Shared Memory : area memori yang dapat diakses bersama oleh beberapa proses. Metode ini lebih cepat karena tidak perlu melalui kernel untuk transfer data.

D. Fungsi System Call Penting

- `Fork ()` : membuat proses anak.
- `Wait ()` : parent menunggu proses anak selesai.
- `Getpid()` dan `getppid()` : menampilkan PID proses dan parentnya.
- `shmget()`, `shmat()`, `shmdt()` : membuat dan mengelola shared memory.

TUJUAN

1. Menerapkan konsep *process management* pada sistem operasi menggunakan bahasa pemrograman C.
2. Menunjukkan kemampuan dalam membuat dan mengelola beberapa *child process* menggunakan `fork()`.
3. Menerapkan metode *Interprocess Communication (IPC)* menggunakan *shared memory*.
4. Menganalisis koordinasi antar proses, waktu eksekusi, dan hasil kerja proses anak.
5. Mengembangkan simulasi sederhana sistem manajemen proses dan aplikasi chat berbasis IPC.

Tugas 1

Sistem manajemen Proses sederhana

Kode program (tugas1_manajemen_proses.c)

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>

void sorting_task() {
    int data[5] = {5, 2, 4, 1, 3};
    for (int i = 0; i < 5; i++) {
        for (int j = i + 1; j < 5; j++) {
            if (data[i] > data[j]) {
                int temp = data[i];
                data[i] = data[j];
                data[j] = temp;
            }
        }
    }
    printf("Child 1 (Sorting) selesai. Hasil: ");
    for (int i = 0; i < 5; i++) printf("%d ", data[i]);
    printf("\n");
}

void searching_task() {
    int data[5] = {10, 20, 30, 40, 50};
    int cari = 30;
    for (int i = 0; i < 5; i++) {
        if (data[i] == cari) {
            printf("Child 2 (Searching) menemukan angka %d di indeks %d\n", cari, i);
            return;
        }
    }
}
```

```

    }
}
printf("Child 2 (Searching) tidak menemukan angka %d\n", cari);
}

void calculation_task() {
    int hasil = 0;
    for (int i = 1; i <= 5; i++) hasil += i;
    printf("Child 3 (Calculation) selesai. Hasil penjumlahan = %d\n", hasil);
}

int main() {
    pid_t pid;
    clock_t start, end;

    printf("Parent PID: %d\n", getpid());
    start = clock();

    for (int i = 0; i < 3; i++) {
        pid = fork();

        if (pid == 0) {
            if (i == 0) sorting_task();
            else if (i == 1) searching_task();
            else calculation_task();

            printf("Child %d (PID: %d) selesai.\n", i + 1, getpid());
            return 0;
        }
    }
}

```

```

for (int i = 0; i < 3; i++) wait(NULL);

end = clock();
double total_time = ((double)(end - start)) / CLOCKS_PER_SEC;
printf("\nParent: Semua proses selesai. Waktu eksekusi total: %.4f detik\n", total_time);
return 0;
}

```

```

rafli_musthofa@RAFLI:~$ gcc tugas1_manajemen_proses.c -o tugas1_manajemen_proses
rafli_musthofa@RAFLI:~$ ./tugas1_manajemen_proses
Parent PID: 3060
Child 1 (Sorting) selesai. Hasil: 1 2 3 4 5
Child 1 (PID: 3061) selesai.
Child 2 (Searching) menemukan angka 30 di indeks 2
Child 2 (PID: 3062) selesai.
Child 3 (Calculation) selesai. Hasil penjumlahan = 15
Child 3 (PID: 3063) selesai.

Parent: Semua proses selesai. Waktu eksekusi total: 0.0006 detik
rafli_musthofa@RAFLI:~$

```

Analisis:

- Parent membuat 3 proses anak menggunakan fork().
- Masing-masing anak menjalankan tugas berbeda:
Pengurutan nilai, pencarian nilai, dan perhitungan angka.
- Parent menunggu semua anak selesai dengan wait().
- Dengan demikian anak Adalah pekerja independent. Pengguna clock() berfungsi untuk menghitung waktu eksekusi total dari seluruh proses.

TUGAS 2

1. Aplikasi chat sederhana dengan shared memory

Kode program file 1 (chat_writer.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ipc.h>
#include <sys/shm.h>

#define SHM_SIZE 1024

int main() {
    key_t key = ftok("chatfile", 65);
    int shmid = shmget(key, SHM_SIZE, 0666 | IPC_CREAT);
    char *shared_mem = (char*) shmat(shmid, NULL, 0);
    char msg[100];

    printf("=== CHAT ROOM (Writer) ===\n");
    while (1) {
        printf("User1> ");
        fgets(msg, sizeof(msg), stdin);
        strcpy(shared_mem, msg);
        if (strncmp(msg, "exit", 4) == 0)
            break;
    }

    shmdt(shared_mem);
    return 0;
}
```

2. Kode program file 2 (chat_reader.c)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <unistd.h>

#define SHM_SIZE 1024

int main() {
    key_t key = ftok("chatfile", 65);
    int shmid = shmget(key, SHM_SIZE, 0666 | IPC_CREAT);
    char *shared_mem = (char*) shmat(shmid, NULL, 0);

    printf("=== CHAT ROOM (Reader) ===\n");
    char last_msg[100] = "";

    while (1) {
        if (strcmp(last_msg, shared_mem) != 0) {
            printf("User2> %s", shared_mem);
            strcpy(last_msg, shared_mem);
            if (strncmp(shared_mem, "exit", 4) == 0)
                break;
        }
        sleep(1);
    }

    shmdt(shared_mem);
    shmctl(shmid, IPC_RMID, NULL);
    return 0;
}
```


Analisis:

Aplikasi chat sederhana ini menggunakan shared memory agar dua proses dapat saling berbagi data pesan.

- Proses writer berperan sebagai pengirim pesan.
- Proses reader membaca isi shared memory secara berkala. Kelebihan shared memory Adalah kecepatan komunikasi data. Kekurangannya, sinkronisasi manual (seperti mutex atau semaphore) diperlukan untuk aplikasi nyata agar tidak terjadi bentrok tulis/baca.

```
rafli_musthofa@RAFLI:~$ ./chat_writer
=== CHAT ROOM (Writer) ===
User1> lelah itu pasti tapi menyerah bukan solusi
User1> 
```

```
rafli_musthofa@RAFLI:~$ ./chat_reader
=== CHAT ROOM (Reader) ===
User2> lelah itu pasti tapi menyerah bukan solusi

```

KESIMPULAN

1. Sistem operasi memungkinkan pembuatan dan koordinasi banyak proses menggunakan `fork()` dan `wait()`.
2. Melalui percobaan Tugas 1, konsep dasar manajemen proses dapat diamati dengan jelas, di mana parent berfungsi sebagai pengatur dan anak sebagai pelaksana tugas.
3. Komunikasi antar proses dapat dilakukan dengan berbagai metode IPC; salah satunya adalah *shared memory* yang efisien dan cepat.
4. Pada Tugas 2, *shared memory* berhasil digunakan untuk membuat simulasi *chat room* sederhana yang memperlihatkan mekanisme komunikasi dua arah antar proses.
5. Kedua tugas ini memberikan pemahaman mendasar mengenai bagaimana sistem operasi mengelola dan menghubungkan proses-proses yang berjalan secara bersamaan.